

Topological Complexity in Deep Neural Networks

Jake Merry

December 10, 2025

1 Introduction and Motivation

Following the work of Naitzat et al., we study the question of what the shape of a data set can tell us about its classification process. The paper *Topology of Deep Neural Networks* provides an in depth dive into how neural networks are able to change the topology of a data set as it passes through its layers. After extensive tests on real and simulated data sets, their conclusions were four-fold: (i) the effectiveness of an activation function is not dependent on whether or not it is smooth, but rather on whether or not it is homeomorphic (a topology preserving map), (ii) neural networks change topology on a layer-by-layer basis until the final layer when the decision boundary becomes a hyperplane, (iii) topological changes occur primarily in the later layers of the network, whereas the earliest layers change only geometry, and (iv) topological changes are robust over multiple training instances. These results give conclusions to questions about why ReLU is a more effective activation function than sigmoidal activations such as tanh and how neural networks are able to reduce highly complex data sets into separated components.

In this paper, after giving a brief exposition on the topology and homology concepts necessary to understand these contributions, we will describe the process taken to conduct such experiments and we will provide results from our own attempts at replicating similar analyses.

2 Background

2.1 Topological Complexity

Let $M \subset \mathbb{R}^d$ be an d -dimensional topological manifold and fix $k \in \{0, 1, \dots, d-1\}$. The tool we select for the analysis of our manifold is the Betti numbers, $\beta(M) = (\beta_0(M), \beta_1(M), \dots, \beta_{d-1}(M))$, which are a topological invariant that will help us track the topology of our manifold as it passes through the layers of a trained network. We will formally define Betti numbers soon, but roughly speaking, the k -th Betti number gives the number of k -dimensional holes of M .

We define the topological complexity, ω , of M as

$$\omega(M) = \sum_{k=0}^{d-1} \beta_k.$$

Simply, $\omega(M)$ gives the total number of holes in the manifold M and we can think of a manifold as being more complex the more holes it has. Our goal here is to observe the changes in Betti numbers as it passes through each layer, ν_j , of an l -layer neural network:

$$\beta(M) \rightarrow \beta(\nu_1(M)) \rightarrow \cdots \rightarrow \beta(\nu_l(M)).$$

Know that in practice, we do not have access to the true underlying manifold of our data set and must estimate the Betti numbers using a point-cloud data set. To do this we will make use of tools from persistent homology, described in the coming sections. Persistent homology is a now common tool in topological data analysis, but note that our goal in this paper differs from that of traditional topological data analysis in that the latter is primarily concerned with studying the shape of the data set itself, whereas we are concerned with manipulating the shape of the data as it passes through our model.

2.2 Homology and Simplicial Homology

Homology is a branch of algebraic topology in which we study the homology of a mathematical structure called a chain complex. This results in a set of Abelian groups called homology groups. The branch of homology itself is vast and complex, so we here provided a simplified exposition on the subject that will work specifically for our own purposes. We will work over the field $\mathbb{F}_2 = \{0, 1\}$. For the $\mathbb{F}_2$ -vector spaces $C_0, C_1, \dots, C_d$, we define the linear boundary operators $\partial_k : C_k \rightarrow C_{k-1}$ satisfying $\partial_k \circ \partial_{k+1} = 0$ for all $k \in \{0, 1, \dots, d\}$. We can then form the chain complex

$$0 \xrightarrow{\partial_{d+1}} C_d \xrightarrow{\partial_d} C_{d-1} \xrightarrow{\partial_{d-1}} \cdots \xrightarrow{\partial_{k+1}} C_k \xrightarrow{\partial_k} C_{k-1} \xrightarrow{\partial_{k+1}} \cdots \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0 \xrightarrow{\partial_0} 0$$

The k -th homology group of this chain complex is the quotient vector space $H_k := \ker(\partial_k) / \text{im}(\partial_{k+1})$ and the elements of H_k are called homology classes. Then, we can define the k -th Betti number of the manifold M as $\beta_k := \dim(H_k) = \text{nullity}(\partial_k) - \text{rank}(\partial_{k+1})$.

With this as motivation, we aim to define vector spaces such as those described previously. To do this we introduce some concepts from simplicial homology. Take an abstract simplicial complex K and define the set $K^{(k)}$ of all k -dimensional simplices in K . We then define the $\mathbb{F}_2$ -vector spaces $C_k(K)$ by the basis $K^{(k)}$. Here, elements of $C_k(K)$ take the form

$$\sum_{j: \sigma_j \in K^{(k)}} n_j \sigma_j, \quad n_j \in \mathbb{F}_2.$$

On a k -simplex, $\sigma = [v_0, \dots, v_k]$ the boundary operators are defined by

$$\partial_k \sigma = \sum_{j=0}^k [v_0, \dots, \hat{v}_j, \dots, v_k],$$

where $\hat{v}_j$ signifies that v_j is missing from the simplex. Then, on $C_k(K)$, we have that

$$\partial_k \left(\sum_{j: \sigma_j \in K^{(k)}} n_j \sigma_j \right) = \sum_{j: \sigma_j \in K^{(k)}} n_j \partial_k \sigma_j.$$

It is worth noting here that working over $\mathbb{F}_2$ forces the relation $H_k(K) \cong \mathbb{F}_2^{\beta_k}$.

Now, given any two abstract simplicial complexes, K_1 and K_2 , we can define a simplicial map $f : K_1^{(0)} \rightarrow K_2^{(0)}$ by $f[v_0, \dots, v_k] = [f(v_0), \dots, f(v_k)]$. Simplicial mappings induce linear mappings $f : C_k(K_1) \rightarrow C_k(K_2)$, defined by

$$f \left(\sum_{j: \sigma_j \in K^{(k)}} n_j \sigma_j \right) = \sum_{j: \sigma_j \in K^{(k)}} n_j f(\sigma_j),$$

and these linear mappings induce a map between homologies, $H_k(f) : H_k(K_1) \rightarrow H_k(K_2)$ by

$$f \left(\left\| \sum_{j: \sigma_j \in K^{(k)}} n_j \sigma_j \right\| \right) = \left\| \sum_{j: \sigma_j \in K^{(k)}} n_j f(\sigma_j) \right\|, \quad \forall k \in \{0, 1, \dots, d+1\}.$$

Composing two simplicial maps gives another simplicial map, following the property of functoriality, $H_k(g \circ f) = H_k(g) \circ H_k(f)$.

Hence, given a point cloud data set $X \subset M$, we wish to construct a simplicial complex K with $K^{(0)} = X$. One way to do this, and the way most adept for this paper, is the Vietoris-Rips complex. Letting δ be a metric on $\mathbb{R}^d$, the Vietoris-Rips complex at scale $\varepsilon > 0$ on X is defined to be

$$\text{VR}_\varepsilon(X) = \{[x_0, \dots, x_k] : \delta(x_i, x_j) \leq 2\varepsilon, i, j = 0, 1, \dots, k, x_0, \dots, x_k \in X, k = 0, 1, \dots, n\}.$$

Our cloud point data set has an underlying topological manifold, i.e. $X \subset M$. If the data set is dense enough, the topology $\text{VR}_\varepsilon(X)$, at a sufficiently small scale, recovers the true topology of the underlying manifold.

Note that the definition of the Vietoris-Rips complex depends on the choice of the scale ε and the metric δ . The best choice of δ is not the Euclidean distance, but rather the geodesic distance on M which we will estimate later. To estimate the proper scale, ε , we introduce the topic of persistent homology.

2.3 Persistent Homology

Homology classes are sensitive to perturbations in data which affects the estimation of Betti numbers. Persistent homology addresses this issue by finding features of the data set that are robust to noise. To do this, we incorporate the scale, $\varepsilon > 0$, into our homology calculations. For instance, when $\varepsilon = 0$, we have that $\text{VR}_0(X) = \{[x] : x \in X\}$, $\beta_0 = |X|$, and $\beta_k = 0$ for all $k > 0$, i.e. $\varepsilon = 0$ overfits the data set. As well, as $\varepsilon \rightarrow \infty$, all of our vertices become connected and we are left with a single contractible space. Hence, the topologies at both extrema are trivial and we seek to find a balance somewhere inbetween. This is done using persistent bar codes.

In short, persistent bar codes are intervals $[\varepsilon, \varepsilon')$, where ε is the scale at which a feature appears and ε' is the scale at which the same feature disappears. We call the length of the interval the persistence of the feature, describing the prominence of the feature. For our purposes, the features that we will be interested in are the the homology classes in the k -th homology group. This allows us to identify an optimal scale ε_* at which all prominent features are present in the complex. To simplify matters, we will fix a finite, ordered list $\varepsilon_0 < \varepsilon_1 < \dots < \varepsilon_m$ from which we will decide our optimal ε_* , but this list could be as large as we like.

We define $K_j = \text{VR}_{\varepsilon_j}(X)$ for all $j = 0, 1, \dots, m$. Then,

$$K_0 \subset K_1 \subset \dots \subset K_m$$

and we can define the inclusion maps $f_j : K_j \hookrightarrow K_{j+1}$ for $j = 0, 1, \dots, m-1$ where each simplex in K_j is sent to the same simplex in K_{j+1} . We can see that f_j is a simplicial map, so it induces the linear maps $H_k(f) : H_k(K_j) \rightarrow H_k(K_{j+1})$. We then have a persistent, or two-dimensional, chain complex as seen below:

$$\begin{array}{cccccccccccccccccccc}
0 & \xrightarrow{\partial_{d+1}} & C_d(K_1) & \xrightarrow{\partial_d} & C_{d-1}(K_1) & \xrightarrow{\partial_{d-1}} & \dots & \xrightarrow{\partial_{k+1}} & C_k(K_1) & \xrightarrow{\partial_k} & C_{k-1}(K_1) & \xrightarrow{\partial_{k+1}} & \dots & \xrightarrow{\partial_2} & C_1(K_1) & \xrightarrow{\partial_1} & C_0(K_1) & \xrightarrow{\partial_0} & 0 \\
& & \downarrow f_1 & & \downarrow f_1 & & & & \downarrow f_1 & & \downarrow f_1 & & & & \downarrow f_1 & & \downarrow f_1 & & \\
0 & \xrightarrow{\partial_{d+1}} & C_d(K_2) & \xrightarrow{\partial_d} & C_{d-1}(K_2) & \xrightarrow{\partial_{d-1}} & \dots & \xrightarrow{\partial_{k+1}} & C_k(K_2) & \xrightarrow{\partial_k} & C_{k-1}(K_2) & \xrightarrow{\partial_{k+1}} & \dots & \xrightarrow{\partial_2} & C_1(K_2) & \xrightarrow{\partial_1} & C_0(K_2) & \xrightarrow{\partial_0} & 0 \\
& & \downarrow f_2 & & \downarrow f_2 & & & & \downarrow f_2 & & \downarrow f_2 & & & & \downarrow f_2 & & \downarrow f_2 & & \\
0 & \xrightarrow{\partial_{d+1}} & C_d(K_3) & \xrightarrow{\partial_d} & C_{d-1}(K_3) & \xrightarrow{\partial_{d-1}} & \dots & \xrightarrow{\partial_{k+1}} & C_k(K_3) & \xrightarrow{\partial_k} & C_{k-1}(K_3) & \xrightarrow{\partial_{k+1}} & \dots & \xrightarrow{\partial_2} & C_1(K_3) & \xrightarrow{\partial_1} & C_0(K_3) & \xrightarrow{\partial_0} & 0 \\
& & \downarrow f_3 & & \downarrow f_3 & & & & \downarrow f_3 & & \downarrow f_3 & & & & \downarrow f_3 & & \downarrow f_3 & & \\
\vdots & & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & & \vdots \\
& & \downarrow f_{m-1} & & \downarrow f_{m-1} & & & & \downarrow f_{m-1} & & \downarrow f_{m-1} & & & & \downarrow f_{m-1} & & \downarrow f_{m-1} & & \\
0 & \xrightarrow{\partial_{d+1}} & C_d(K_m) & \xrightarrow{\partial_d} & C_{d-1}(K_m) & \xrightarrow{\partial_{d-1}} & \dots & \xrightarrow{\partial_{k+1}} & C_k(K_m) & \xrightarrow{\partial_k} & C_{k-1}(K_m) & \xrightarrow{\partial_{k+1}} & \dots & \xrightarrow{\partial_2} & C_1(K_m) & \xrightarrow{\partial_1} & C_0(K_m) & \xrightarrow{\partial_0} & 0
\end{array}$$

However, this structure is clearly very complex and is not intuitive to follow. Thus, we make use of a result from (Zomorodian and Carlsson, 2005), which states that a persistent barcode completely describes the persistent complex above. Therefore, we focus our attention on said persistent barcodes instead. Additionally, we can compute persistent barcodes without actually computing homology at every scale we test. We can instead define the p -persistent k -th homology group by

$$H_k^{j,p} = \ker(H_k(K_j)) / (\text{im}(H_k(K_{j+p})) \cap \ker(H_k(K_j))),$$

which capture the homology classes in $H_k(K_j)$ that persist in $H_k(K_{j+p})$. This allows us to track the persistence of each homology class as it passes through the chain and obtain the associated persistent barcodes.

3 Experimental Process and Results

We begin by noting that there is a vast computational difference for data simulated from a known manifold and for real-world data. Real-world is evermore complex and can not be entirely separated between categories, making it nearly impossible to simplify. Hence, a majority of analysis in (Naitzat) was conducted on simulated data and results were then validated using real-world data, after the fact. In Section 4, we will conduct analysis on two simulated data sets and one real-world data set for a sparser result.

As well, Naitzat et al.'s focus was on what they considered “well-trained” neural networks. This is one in which the training error is zero and the generalization error is ≈ 0.01 for simulated data and about 3 – 5 for real-world data. These types of networks are, however, very difficult and time consuming to train, making them impractical for many purposes. Thus, we will not focus on as strict of a model and instead try to generalize Naitzat et al.'s results to less “well-trained” networks.

Our experiment consists of four major steps:

- (i) generate a manifold $M = M_a \cup M_b \subset \mathbb{R}^d$, $M_a \cap M_b = \emptyset$,
- (ii) uniformly sample a dense point cloud $X \subset M$ with $X_i = X \cap M_i$, $i = a, b$,
- (iii) train an l -layer neural network $\nu : \mathbb{R}^d \rightarrow [0, 1]$ on the labeled training set $X = X_a \cup X_b$ to classify points on M ,
- (iv) compute homology of the output at the j -th layer $\nu_j(X_i)$, $j = 0, 1, \dots, l + 1$.

Computing homology is by far the most computationally expensive step in our procedure. To start, we must select the metric δ and scale ε to construct Vietoris-Rips complex on X that accurately describes the topology of M . The best choice of δ is the graph geodesic distance on the K -nearest neighbors graph of X . We denote this metric by δ_k and note that, while the Euclidean distance is formed to construct the KNN graph and find δ_k , this is its only use here and we now rely on δ_k exclusively for this computations.

This leaves us with two parameters to compute, k_* and ε_* . However, this is much more difficult than estimating one parameter, specifically since there is no multivariable analog of persistent barcodes. We instead construct a filtered complex over k and ε that help us select optimal values. Our exact procedure for calculating homologies is as follows:

1. Set $\varepsilon = 1$. Find k_* such that $\beta_0(\text{VR}_{k_*, 1}(X)) = \beta_0(M)$.
2. Define δ_{k_*} and find ε_* such that $\beta_i(\text{VR}_{k_*, \varepsilon_*}(X)) = \beta_i(M)$, $i = 1, 2$, i.e. find the correct scale.
3. Estimate $\beta(M) \approx \beta(\text{VR}_{k_*, \varepsilon_*}(X))$

Note that if there are a range of values that satisfy the condition of k_* or ε_* , then we select the middle most value in the range.

(Naitzat) used three simulated data sets trained on over 1,200 total neural networks and computed over 10,000 homologies to conclude that, not only do neural networks change the topology of a data set on a layer-by-layer-basis as it passes through, but that non-homeomorphic activations like ReLU change topology at a faster rate than non-homeomorphic mappings. The authors also used varying networks widths and depths and concluded that width does not significantly affect topological simplifications but does cause the network to be harder to train, and that the depth is an important factor in that topological changes in deeper networks are spread evenly over many layers, while in shallow networks, the final layers are forced to do all of the simplification. These conclusion were tested on a smaller set of real-world data samples and all tests validated the conclusions from the simulated data examples.

It should be noted that this experiment can be stated in the simple format: for a manifold $M = M_a \cup M_b$ with $M_a \cap M_b = \emptyset$, we track how the decision boundary simplifies throughout the network, going from a very topologically complicated object in the original training set to a linear hyperplane by the last layer.

4 Limitations and Obstacles

The results given by Naitzat et al. shed an important light not only on the inner workings of how neural networks are able to work as classifier models for extremely complicated data sets, but also on how different model parameters function to make these changes happen. However, these experiments are limited in how much they are able to accomplish. For one, while training neural networks to near perfect accuracy is an extremely hard problem to begin with, it hardly carries the

bulk of the computational effort required for tracking this topological changes. The authors shed light on the fact that, even working on very powerful machinery, the computation of a single pair of parameters (k_*, ε_*) can take over 80 minutes. This makes our job of tracking many of them per model very difficult. In fact, in our attempts at producing similar analyses on our own data sets, we were unable to obtain coherent results these calculations. The R code we used can be found in the Appendix at this end of this paper, but they do not seem to be of significant use other than the fact that every network that we trained with ReLU fit to a higher training and testing accuracy than those fitted with the same parameters but with tanh activation. This, however, only leaves us with the original question of why ReLU works more efficient, it does not explain how.

Our main obstacle in computation was using `TDastats` to compute persistent barcodes. The discontinuous nature of the graph geodesic distance, δ_k , and the fact that the `compute_homology` function requires a finite threshold both add to the result that homology intervals get cut back and those features which should persist indefinitely do not, i.e. our Betti number computations continually lead to β_0 , a topologically impossible feature. In many instances, the largest death time in the 0-dimensional persistent was very near the threshold, causing all 0-dimensional classes to appear extinct near the end of the range.

5 Discussion and Interpretation

Although the numerical homology computations were unstable, we can still describe the expected behavior of the experiment. For our simulated data set, we used a ball-shell manifold, where M_a was the unit ball in $\mathbb{R}^3$ and M_b was a set from radius 1 to 2, surrounding M_a . Our manifold $M = M_a \cup M_b$ has Betti numbers $\beta(M) = (1, 0, 0)$, i.e. it is a contractible space, and as such we would expect that at the correct scale (k_*, ε_*) , the Vietoris-Rips complex should correctly exhibit a manifold with one connected component and no holes of any higher dimension. For our simulated data, we selected the fashion MNIST data set. Both data sets were split into training and testing sets and both training sets we fit to neural networks with tanh and ReLU activations. In almost all cases, the training and testing error of the ReLU networks were lower than those of the tanh activated networks with the mean accuracies summarized in the table below:

<u>tanh</u>			<u>ReLU</u>		
	Simulated	Real		Simulated	Real
Training	99.09	93.15	Training	99.61	95.69
Testing	99.20	91.97	Testing	99.05	92.87

If this data set were to pass through a well-trained neural network, we would assume that in the earlier layers, Betti numbers would only change slightly, whereas in the later layers, the data set would fold into $\mathbb{R}^4$ to collapse the two regions onto themselves and create a linearly separable manifold.

6 Conclusion

In this project, we successfully simulated from a selected manifold and trained neural networks to fit real and simulated data sets. Although our homology computations were not stable enough to obtain reliable results about our data, the issues we encountered reflect real challenges faced in analyzing data topologically. Specifically, we displayed the difficulty in retrieving information about population data when we are only given a sample to work with.

Naitzat et al.'s conclusions are about to bring about clarity to some aspect about neural networks that mathematicians have wondered about for years. Specifically, it explains that ReLU outperforms other activations due to the fact that it is a non-homeomorphic mapping and not because it is a non-smooth mapping. As well as explaining how neural networks are able to classify complex data sets.

The paper does, however, leave some questions unanswered. For instance, the authors speculate, while they did not study the mechanisms of how ReLU actually changes topology, that since an affine map can only cause a rotation, reflection, or a contraction, that ReLU gives an activation function the ability to fold space in a higher dimension. However, this is not proven and could, whether true or false, bring more clarity to the inner workings of neural networks.

Despite the challenges faced, this project illustrates the core principles of topological learning that neural networks continuously deform the shape of our data and persistent homology provides a way to measuring these deformations.

Appendix

Useful Definitions

Definition. (Topology)

Let X be a set. A topology, τ , on X is a collection of subsets of X satisfying the following:

- (i) $\emptyset, X \in \tau$;
- (ii) $X_1, \dots, X_n \in \tau \implies \bigcap_{k=1}^n X_k \in \tau$;
- (iii) $\{X_k\} \subset \tau \implies \bigcup_k X_k \in \tau$.

Definition. (Topological Space)

An ordered pair (X, τ) is said to be a topological space if X is a set and τ is a topology on X .

Definition. (Topological Manifold)

Roughly, an n -dimensional topological manifold is a topological space that locally resembles $\mathbb{R}^n$ at each point. Formally, it is a second countable Hausdorff space that is locally homeomorphic to $\mathbb{R}^n$.

Definition. (Contractible)

A topological space is called contractible if it can be continuously deformed to a single point.

Definition. (Simplex)

An n -dimensional simplex is the convex hull of a set of $n + 1$ vertices.

Definition. (Simplicial Complex)

Let K be a finite collection of simplices of dimension at most m . We say K is a finite, m -dimensional simplicial complex if the following conditions are satisfied:

- (i) if $\sigma_0 \subset \sigma \in K$, then $\sigma_0 \in K$;
- (ii) if $\sigma = \sigma_1 \cap \sigma_2 \neq \emptyset$ for $\sigma_1, \sigma_2 \in K$, then $\sigma \subset \sigma_1$ and $\sigma \subset \sigma_2$.

R Code Used for Analysis

```

library(keras3)
library(FNN)
library(igraph)
library(TDAstats)

set.seed(123)

`%<-%` <- keras3::`%<-%`

# Simulated data sampling
sample_dirs3 <- function(n) { # sample directions in  $R^3$ 
  dirs <- matrix(rnorm(3*n), nrow=n, ncol=3)
  norms <- sqrt(rowSums(dirs^2))
  return(dirs/norms)
}

sample_ball <- function(n, r=1) { #  $M_a$  = unit ball
  u_list <- runif(n)
  r_list <- r * u_list^(1/3)

  dirs <- sample_dirs3(n)
  return(dirs * r_list)
}

sample_shell <- function(n, r_inner=1, r_outer=2) { #  $M_b$  = outer shell between  $r=1,2$ 
  u_list <- runif(n)
  r_list <- ((r_outer^3 - r_inner^3) * u_list + r_inner^3)^(1/3)

  dirs <- sample_dirs3(n)
  return(dirs * r_list)
}

sample_manifold <- function(n, frac_a=0.35) {
  n_a <- round(n * frac_a)
  n_b <- n - n_a

  X_a <- sample_ball(n_a, 1)
  y_a <- rep(0, times=n_a)

  X_b <- sample_shell(n_b, 1, 2)
  y_b <- rep(1, times=n_b)

  X <- rbind(X_a, X_b)
  y <- factor(c(y_a, y_b), levels=c(0, 1))

  return(list(X=X, y=y))
}

```

```

# Simulated data : M a sphere
beta_M <- c(1, 0, 0)
c(X, y) %<-% sample_manifold(10000)

n <- nrow(X)
rand_idx <- sample(1:n)
train_frac <- 0.8
n_train <- floor(train_frac * n)

train_idx <- rand_idx[1:n_train]
test_idx <- rand_idx[(n_train + 1):n]

X_train <- as.matrix(X[train_idx, ]); y_train <- as.integer(y[train_idx]) - 1
X_test <- as.matrix(X[test_idx, ]); y_test <- as.integer(y[test_idx]) - 1

# Real data
fashion_mnist <- dataset_fashion_mnist()
c(train_data_real, train_labels_real) %<-% fashion_mnist$train
c(test_data_real, test_labels_real) %<-% fashion_mnist$test

train_data_real <- array_reshape(train_data_real, c(dim(train_data_real)[1], 28, 28, 1)) / 255
test_data_real <- array_reshape(test_data_real, c(dim(test_data_real)[1], 28, 28, 1)) / 255
train_labels_real <- to_categorical(train_labels_real, 10)
test_labels_real <- to_categorical(test_labels_real, 10)

epochs <- 50
batch_size <- 128

# Simulated NN - tanh
model_sim_tanh <- keras_model_sequential(input_shape=3) %>%
  layer_dense(units=32, activation="tanh") %>%
  layer_dense(units=32, activation="tanh") %>%
  layer_dense(units=1, activation="sigmoid")

model_sim_tanh %>% compile(optimizer=optimizer_adam(learning_rate=1e-3),
  loss="binary_crossentropy",
  metric="accuracy")

model_sim_tanh %>% fit(X_train, y_train, epochs=epochs, batch_size=batch_size, verbose=1)

score <- model_sim_tanh %>% evaluate(X_test, y_test, verbose=0)
score

# Simulated NN - ReLU
model_sim_relu <- keras_model_sequential(input_shape=3) %>%
  layer_dense(units=32, activation="relu") %>%
  layer_dense(units=32, activation="relu") %>%
  layer_dense(units=1, activation="sigmoid")

```

```

model_sim_relu %>% compile(optimizer=optimizer_adam(learning_rate=1e-3),
                           loss="binary_crossentropy",
                           metric="accuracy")

model_sim_relu %>% fit(X_train, y_train, epochs=epochs, batch_size=batch_size, verbose=1)

score <- model_sim_relu %>% evaluate(X_test, y_test, verbose=0)
score

```

Real CNN - tanh

```

model_real_tanh <- keras_model_sequential(input_shape=c(28, 28, 1)) %>%
  layer_conv_2d(filters=32, kernel_size=c(3, 3), activation="tanh") %>%
  layer_conv_2d(filters=32, kernel_size=c(3, 3), activation="tanh") %>%
  layer_max_pooling_2d(pool_size=c(2, 2)) %>%
  layer_dropout(0.25) %>%

  layer_conv_2d(filters=64, kernel_size=c(3, 3), activation="tanh") %>%
  layer_conv_2d(filters=64, kernel_size=c(3, 3), activation="tanh") %>%
  layer_max_pooling_2d(pool_size=c(2, 2)) %>%
  layer_dropout(0.25) %>%

  layer_flatten() %>%
  layer_dense(units=256, activation="tanh") %>%
  layer_dropout(0.5) %>%
  layer_dense(units=10, activation="softmax")

model_real_tanh %>% compile(optimizer=optimizer_adam(learning_rate=1e-3),
                           loss="categorical_crossentropy",
                           metric="accuracy")

model_real_tanh %>% fit(train_data_real, train_labels_real,
                      epochs=epochs, batch_size=batch_size, verbose=1)

score <- model_real_tanh %>% evaluate(test_data_real, test_labels_real, verbose=0)
score

```

Real CNN - ReLU

```

model_real_relu <- keras_model_sequential(input_shape=c(28, 28, 1)) %>%
  layer_conv_2d(filters=32, kernel_size=c(3, 3), activation="relu") %>%
  layer_conv_2d(filters=32, kernel_size=c(3, 3), activation="relu") %>%
  layer_max_pooling_2d(pool_size=c(2, 2)) %>%
  layer_dropout(0.25) %>%

  layer_conv_2d(filters=64, kernel_size=c(3, 3), activation="relu") %>%
  layer_conv_2d(filters=64, kernel_size=c(3, 3), activation="relu") %>%
  layer_max_pooling_2d(pool_size=c(2, 2)) %>%
  layer_dropout(0.25) %>%

```

```

layer_flatten() %>%
layer_dense(units=256, activation="relu") %>%
layer_dropout(0.5) %>%
layer_dense(units=10, activation="softmax")

model_real_relu %>% compile(optimizer=optimizer_adam(learning_rate=1e-3),
                           loss="categorical_crossentropy",
                           metric="accuracy")

model_real_relu %>% fit(train_data_real, train_labels_real,
                      epochs=epochs, batch_size=batch_size, verbose=1)

score <- model_real_relu %>% evaluate(test_data_real, test_labels_real, verbose=0)
score

# Simulated homology computations
KNN <- function(X, k) {
  n <- nrow(X)
  nn <- FNN::get.knn(X, k=k)

  edges <- c()
  for (i in 1:n) {
    for (j in nn$nn.index[i, ]) {
      if (j != i) edges <- rbind(edges, c(i, j))
    }
  }

  # remove duplicates
  edges <- unique(t(apply(edges, 1, sort)))

  return(igraph::graph_from_edgelist(edges, directed=FALSE)) # KNN graph
}

beta0_vr_k1 <- function(X, k) {
  graph <- KNN(X, k)
  return(igraph::components(graph)$no) # int - num components
}

delta_k <- function(X, k) {
  graph <- KNN(X, k)
  D <- igraph::distances(graph)
  D[!is.finite(D)] <- Inf
  return(as.matrix(D)) # matrix - distances
}

compute_betti <- function(ph, eps, max_dim=2) {
  betti <- integer(max_dim + 1)

```

```

for (d in 0:max_dim) {
  rows <- ph[ph$dimension == d, ]
  if (nrow(rows) == 0) betti[d+1] <- 0
  else {
    alive <- rows$birth <= eps & (rows$death >= eps | is.infinite(rows$death))
    betti[d+1] <- sum(alive)
  }
}
print(betti)
return(betti) # list - betti numbers
}

estimate_vr_parameters <- function(X, beta_M, k_vals, eps_vals, max_dim=2) {

  # Step 1 : find k_opt
  beta0_vals <- sapply(k_vals, function(k) beta0_vr_k1(X, k)) # eps = 1

  k_good <- k_vals[beta0_vals == beta_M[1]]
  if (length(k_good) == 0) stop("No k in list gives correct zeroth homology.")

  k_opt <- k_good[ceiling(length(k_good) / 2)]

  # Step 2 : find eps_opt
  k <- k_opt

  D_k <- delta_k(X, k_opt)

  eps_good <- c()
  betti_list <- list()
  ph <- TDASTATS::calculate_homology(mat=D_k, dim=max_dim, threshold=max(eps_vals),
                                     format="distmat", return_df=TRUE)

  for (eps in eps_vals) {
    betti <- compute_betti(ph, eps, max_dim)
    if (all(betti[c(2, 3)] == beta_M[c(2, 3)])) {
      eps_good <- c(eps_good, eps)
      betti_list[[length(betti_list)+1]] <- betti
    }
  }

  if (length(eps_good) == 0) stop("No epsilon in list gives correct homology")
  eps_opt <- eps_good[ceiling(length(eps_good) / 2)]

  return(list(k=k_opt, eps=eps_opt))
}

compute_betti_for_X <- function(X, k_opt, eps_opt, max_dim=2) {

```

```

D_k <- delta_k(X, k_opt)
ph <- TDAstats::calculate_homology(mat=D_k, dim=max_dim, threshold=eps_opt,
                                   format="distmat", return_df=TRUE)
beti_list <- compute_beti(ph, eps_opt, max_dim)

return(beti_list)
}

get_layer_activs <- function(model, X) {
  layer_outputs <- lapply(model$layers, function(layer) layer$output)

  activ_model <- keras_model(inputs=model$inputs, outputs=layer_outputs)
  return(predict(activ_model, X))
}

track_homology <- function(model, X, k_opt, eps_opt, max_dim=2) {
  beti_list <- list()

  beti <- compute_beti_for_X(X, k_opt, eps_opt, max_dim)
  beti_list[[length(beti_list)+1]] <- beti

  activs <- get_layer_activs(model, X)

  for (i in 1:length(activs)) {
    activ <- activs[[i]]

    activ_mat <- matrix(activ, nrow=dim(activ)[1], ncol=prod(dim(activ)[-1]))
    beti <- compute_beti_for_X(activ_mat, k_opt, eps_opt, max_dim)
    beti_list[[length(beti_list)+1]] <- beti
  }

  beti_mat <- do.call(rbind, beti_list)
  return(beti_mat)
}

k_vals <- 3:20
eps_vals <- seq(0.5, 5.5, by=0.5)

X_sub <- X_train[sample(1:nrow(X_train), 100), ] # sample of 500 for homology computations

params <- estimate_vr_parameters(X_sub, beta_M, k_vals, eps_vals, max_dim=2)

beti_sim_tanh <- track_homology(model_sim_tanh, X_sub, params$k, params$eps)
beti_sim_relu <- track_homology(model_sim_relu, X_sub, params$k, params$eps)
beti_sim_tanh; beti_sim_relu

```

References

- Naitzat, G., Zhitnikov, A., & Lim, L.-H. (2020). *Topology of Deep Neural Networks*. Journal of Machine Learning Research, 21(184), 1-40.
- Lee, J. M. (2011). *Introduction to Topological Manifolds* (2nd ed.) Springer.
- Zomorodian, A. & Carlsson, G. (2005). Computing Persistent Homology. Discrete & Computational Geometry, 33, 249-274.